

Tf-idf important doc .

QUESTION:-

Q1 Yeah so basically listern my thing very carefully let term is "odissa train blast " now i have 3 terms right odissa as t1, train as t2, blast as t3 right . Let's find the tf-idf for first term t1 odissa so tf is let's say in doc 1 ->3 okay now idf which is term in whole globally which is 2(therefore $1+\lg(2000/2)$) comes as 4 so tfidf of word odissa is $4*3=12$ right but that's just one component of doc1 . same goes for train , same goes for blast right and we come as doc1=[12 34 10] okay like this array or vectors I am still wanna know why for each word i check tf and it's global presence in doc and multiply them $4*3$ as 12 . should just be checking just tf alone like 3 for doc 1 , train comes 17 times , and blast comes 5 so [3,17,5] why bother with the frequency as well it's global reach/populatiry for a doc to be most appropriate . how does it help me ? explain with this example only pls in order to get more good or depth idea of it.

Got it. Stick with your exact example: **query = “odissa train blast”** (3 terms).

You’re asking: why not just use TF like **[3, 17, 5]** for doc1? Why multiply by IDF at all?

Because **TF alone can’t tell whether a word is informative or “just common everywhere.”** IDF fixes that by down-weighting words that appear in many documents.

Let’s make it real with the same 3 words.

Imagine a news corpus (2000 docs)

- **train** appears in *tons* of news articles (very common)
- **blast** appears in fewer (still common-ish)
- **odissa** appears in very few (very specific)

Say:

- $df(\text{train}) = 1500$ docs
- $df(\text{blast}) = 200$ docs
- $df(\text{odissa}) = 2$ docs

Now compute IDF (your style):

$$idf(t) = 1 + \log\left(\frac{N}{df(t)}\right) \quad idf(t) = 1 + \log(df(t)N)$$

with $N=2000$

Roughly:

- $idf(\text{train}) = 1 + \log(2000/1500) = \text{small (close to 1.x)}$
- $idf(\text{blast}) = 1 + \log(2000/200) = \text{medium}$
- $idf(\text{odissa}) = 1 + \log(2000/2) = \text{big (around 4+ depending on log base)}$

So **odissa** is much more “rare and identifying” than **train**.

Now compare two documents

Doc1 (your example)

TFs:

- odissa = 3
 - train = 17
 - blast = 5
- So TF vector: **[3, 17, 5]**

Doc2 (a generic train article)

TFs:

- odissa = 0
 - train = 40
 - blast = 1
- TF vector: **[0, 40, 1]**

If you rank by **TF sum** for query terms:

- Doc1 score = $3 + 17 + 5 = \mathbf{25}$
- Doc2 score = $0 + 40 + 1 = \mathbf{41}$

TF-only would pick Doc2 as “more relevant” just because it says “train” a lot, even though it doesn’t mention **odissa** at all.

That’s the core problem.

What TF-IDF does here

It says: “train is common, so don’t let it dominate the score.”

Let’s use simple IDFs to illustrate (not exact logs, just realistic relative sizes):

- $\text{idf}(\text{odissa}) = 4$
- $\text{idf}(\text{train}) = 1.1$
- $\text{idf}(\text{blast}) = 2$

Now TF-IDF:

Doc1 TF-IDF

- $\text{odissa: } 3 \times 4 = \mathbf{12}$
- $\text{train: } 17 \times 1.1 = \mathbf{18.7}$
- $\text{blast: } 5 \times 2 = \mathbf{10}$

Vector $\sim [12, 18.7, 10]$

Doc2 TF-IDF

- $\text{odissa: } 0 \times 4 = \mathbf{0}$
- $\text{train: } 40 \times 1.1 = \mathbf{44}$
- $\text{blast: } 1 \times 2 = \mathbf{2}$

Vector $\sim [0, 44, 2]$

Now look what happened:

- Doc2 still has a huge “train” weight, but **it gets punished for missing odissa**, which is the most distinctive term.

When you compare to the query, TF-IDF makes **matching “odissa” matter a lot** because it’s rare in the whole corpus.

The intuition in one sentence

TF tells you “how much this document talks about the word.”

IDF tells you “how special that word is for identifying the right documents.”

So TF-IDF = “how much this document talks about a word that actually helps distinguish the right docs from the rest.”

Why “global popularity” helps your query specifically

For the query “**odissa train blast**”:

- “train” alone matches thousands of docs → not enough to find the right one
- “blast” narrows it somewhat
- “odissa” narrows it massively (because only a few docs mention it)

So IDF makes the system behave like a human would:

“A doc mentioning *Odissa* is probably way more relevant than a doc that just repeats ‘train’.”

If you want, give me df values you’re using (or your real corpus stats), and I’ll compute the exact TF-IDF vectors for doc1 and show how the ranking changes versus TF-only.

but even calculating tf-idf for each term does not tells the righth idea if that doc is most appropriate or not . Like if let's say odissa comes 20 times in doc2 it's $tf \cdot idf$ would be very big as it has odissa and train as well. so how can u even say which doc is more appropaite .

Exactly. **TF-IDF is not a magic “this doc is correct” detector.** It’s just a *term weighting* scheme. To decide which document is more appropriate, you still need a **scoring rule** that compares the *query* to the *document*.

So there are two layers:

1. **TF-IDF** gives you weights for each term inside each doc (and the query).
2. A **similarity / ranking function** uses those weights to rank docs.

If you only look at “TF-IDF for each term” in isolation, you can absolutely get fooled by a doc that repeats “odissa” 20 times.

Let’s use your same query:

Query: odissa train blast

Terms: t1=odissa, t2=train, t3=blast

Assume IDFs roughly (just for intuition):

- $\text{idf}(\text{odissa}) = 4$ (rare)
- $\text{idf}(\text{train}) = 1$ (common)
- $\text{idf}(\text{blast}) = 2$ (medium)

Now two docs:

Doc1 (good match, normal article):

- odissa = 3, train = 17, blast = 5
TF-IDF vector $\approx [3 \cdot 4, 17 \cdot 1, 5 \cdot 2] = [12, 17, 10]$

Doc2 (maybe spammy or repetitive):

- odissa = 20, train = 2, blast = 0
TF-IDF vector $\approx [80, 2, 0]$

If you do the naive “sum TF-IDF components”:

- Doc1 sum = $12 + 17 + 10 = \mathbf{39}$
- Doc2 sum = $80 + 2 + 0 = \mathbf{82}$
Naive sum says Doc2 is better, even though it **misses “blast”** and barely talks about trains.

That’s why people don’t stop at “TF-IDF per term”.

How do we decide which doc is more appropriate then?

A) Match strength should depend on *overlap with the query*

A very common scoring idea is the **dot product** between query vector and doc vector:

$$\text{score}(d,q) = \sum_{t \in q} \text{tfidf}(t,d) \times \text{tfidf}(t,q) \quad \text{score}(d,q) = \left| \sum_{t \in q} \text{tfidf}(t,d) \right| \times \left| \sum_{t \in q} \text{tfidf}(t,q) \right|$$

For your query, the query vector (TF=1 each) is roughly:

$$q \approx [4, 1, 2] \quad q \approx [4, 1, 2]$$

Now scores:

$$\text{Doc1: } [12, 17, 10] \cdot [4, 1, 2] = 12 \cdot 4 + 17 \cdot 1 + 10 \cdot 2 = 48 + 17 + 20 = 85$$

$$\text{Doc2: } [80, 2, 0] \cdot [4, 1, 2] = 80 \cdot 4 + 2 \cdot 1 + 0 \cdot 2 = 320 + 2 + 0 = 322$$

Doc2 still wins here because “odissa” dominates. So dot product alone still can be fooled by repetition.

That leads to the next fix:

B) Normalize so “repeating a lot” doesn’t automatically win

This is why cosine similarity exists:

$$\text{cosine}(d,q) = \frac{d \cdot q}{\|d\| \|q\|} \quad \text{cos}(d,q) = \frac{d \cdot q}{\|d\| \|q\|}$$

Cosine cares about **direction** (topic mix), not raw magnitude.

- Doc1 has a balanced mix of odissa/train/blast
- Doc2 is basically “odissa odissa odissa ...” with missing blast

Cosine will usually favor the doc whose term distribution matches the query better, not just the one with bigger counts.

C) Reduce the impact of huge TF counts

Even before cosine, many systems don't use raw TF. They use **sublinear TF**, like:

$$tf' = 1 + \log_{10}(tf) \quad tf' = 1 + \sqrt{\log(tf)}$$

So $tf=20$ doesn't become "20x stronger" than $tf=1$. It becomes "a bit stronger".

This alone stops the "odessa 20 times" doc from exploding the score.

D) Missing query terms should hurt

Sometimes you add simple rules:

- Require all query terms (AND filter), or
- Penalize missing terms (especially important ones)

For your query, a doc missing **blast** might be less appropriate, so you can enforce that.

So what's the real answer to your concern?

You're right: **TF-IDF weights alone don't tell you which doc is best.**

They only tell you **how important each term is inside that doc.**

To rank documents properly, you combine TF-IDF with:

- a similarity measure (dot product / cosine), and
- normalization and/or sublinear TF (to prevent repetition dominating),
- sometimes rules for missing terms.

That's the full pipeline that makes it work.

If you want, give me **Doc1 and Doc2 term counts** (or short text), and I'll compute:

- naïve sum
- dot product
- cosine (with normalization)

- log-TF version
and you'll see exactly which one picks the more sensible document and why.

so we can overlap based on just frequency also since tf-idf alone could not tell then just try to overlap freq normalized frequency with each ? why goes with tfidf then normalized?

You *can* rank with just TF (or normalized TF). People did that early on, and it works okay for some cases. TF-IDF is used because it fixes a specific problem that TF (even normalized) still can't fix:

TF can't tell the difference between a term that's "useful for identifying the right docs" and a term that's "everywhere."

Normalization mainly fixes **document length** bias.

IDF fixes **term commonness** bias.

They solve different problems.

1) Why normalized TF alone still fails

Say your query is: **"odissa train blast"**

Across a news corpus:

- **train** appears in loads of docs (general word)
- **odissa** appears in very few (specific word)

Now imagine two docs:

- **Doc A:** lots of "train", little/no "odissa"
- **Doc B:** has "odissa" once and "blast" once

Even with normalized TF (like count / doc length), **Doc A can still score well** because "train" is frequent in many documents and you're treating it as equally important as "odissa".

Normalized TF does not say:

"hey, train is common across the whole collection, don't let it dominate."

IDF does.

2) What IDF adds that TF normalization can't

IDF is like an “informativeness weight”:

- If a term appears in **almost every document**, it doesn't help you distinguish relevant vs non-relevant docs → give it low weight.
- If it appears in **very few documents**, it's a strong clue → give it high weight.

So for your query:

- Matching **odissa** should matter more than matching **train**, because “train” matches thousands of unrelated docs.

That's the whole reason TF-IDF exists.

3) Why both TF-IDF and normalization are used together

Because they fix two separate issues:

A) IDF handles term commonness (global)

- stops “train” from dominating just because it appears everywhere.

B) Normalization handles document length / verbosity (local)

- stops long docs (or spammy repetition like “odissa odissa odissa...”) from winning just because they repeat terms more.

So the combo is basically:

- **IDF**: “is this word special across the corpus?”
- **TF (often log-scaled)**: “does this doc emphasize it?”
- **Normalization**: “don't reward length or repetition too much”

4) If you want the simplest “why not just TF normalized?”

Because TF normalized answers:

“What fraction of this doc is made of my query words?”

But TF-IDF answers:

“How strongly does this doc contain the query words that are actually *useful for distinguishing* the right docs from the rest?”

For search, the second one is what you want.

5) Practical note

A lot of modern search engines go beyond TF-IDF and use **BM25**, which is basically:

- TF with diminishing returns (so $tf=20$ isn't 20x)
- IDF
- length normalization baked in

It's the same motivation you're circling around: TF alone isn't enough, normalization alone isn't enough, so you combine signals.

If you want, I can show a tiny numeric example where:

- normalized TF ranks a “train-heavy” doc too high,
- TF-IDF + normalization correctly boosts the “odissa” doc.

but tf normalized and then overlapping with query normalized tf would give the informative weight as well correct cosine isn't?

Thought for 14s

Cosine with **normalized TF** helps, but it *doesn't* automatically give you “informativeness”. It only measures **shape similarity** between the query and the doc using whatever weights you feed it.

If the weights are just TF, then cosine is comparing **term-frequency patterns**, not “how discriminative these terms are across the whole collection”.

Why cosine(TF) is not enough

Cosine answers:

“Do the query and doc use the same words in similar proportions?”

It does **not** know this critical global fact:

- **train** appears in tons of documents

- **odissa** appears in very few

That “global rarity” signal is exactly what **IDF** adds.

Same example: “odissa train blast”

Query terms: [odissa, train, blast]

Query TF vector is [1,1,1] and normalized it’s roughly [0.577, 0.577, 0.577].

Now compare two docs:

Doc A (generic train/blast story, no odissa):

TF = [0, 40, 10]

Normalize it: magnitude = $\sqrt{(0^2+40^2+10^2)}=\sqrt{1700} \approx 41.23$

So normalized doc $\approx$ [0, 0.970, 0.243]

Cosine with query:

$$0*0.577+0.970*0.577+0.243*0.577 \approx (1.213)*0.577 \approx 0.700*0.577 + 0.970*0.577 + 0.243*0.577 \approx (1.213)*0.577 \approx 0.70$$

So even though it **misses “odissa” completely**, it still gets a pretty high similarity (0.70) just because it matches the two common words well.

That’s the problem you’re feeling.

Doc B (actually about odissa train blast):

TF = [3, 17, 5]

Normalize: magnitude $\sqrt{(9+289+25)}=\sqrt{323} \approx 17.97$

Normalized $\approx$ [0.167, 0.946, 0.278]

Cosine:

$$(0.167+0.946+0.278)*0.577 \approx 1.391*0.577 \approx 0.80$$

Doc B wins (0.80 vs 0.70), but notice: **Doc A is still surprisingly competitive** even though it’s missing the most identifying term.

What TF-IDF changes (even with cosine)

If you switch to TF-IDF, the query vector is no longer “equal weight for all words”.

Because in a big corpus:

- $\text{idf}(\text{odissa})$ is high (rare, super informative)
- $\text{idf}(\text{train})$ is low (common, not informative)
- $\text{idf}(\text{blast})$ medium

So the query vector becomes more like:

[high, low, medium]

After normalization, it points **more toward “odissa”**.

Now a doc that has **no odissa** can’t align well with the query direction anymore, so its cosine drops a lot. That’s the whole win: **IDF makes the query “care more” about the rare term**.

So your statement

“tf normalized and then overlap with query normalized tf gives informative weight as well correct cosine isn’t?”

Not really. It gives **length control** and **distribution matching**, but it still treats all terms as equally valuable signals. Cosine can’t invent “informativeness” unless the weights already include it.

- **Normalization/cosine** fixes verbosity/length effects.
- **IDF** fixes “common words shouldn’t dominate” effects.

They’re complementary.

If you want, I can compute one concrete TF-IDF cosine with some sample IDFs (like $\text{idf}(\text{odissa})=4$, $\text{idf}(\text{train})=1.1$, $\text{idf}(\text{blast})=2$) and you’ll see Doc A’s score drop sharply just because it lacks odissa.

okay so even it wins but we need the global comparison in order to have rarity becuz there can be chances that even cosine can beat the approach

Yep, that’s the idea.

Cosine on **TF-only** can still rank the “wrong” doc higher because it has no clue which words are *rare and identifying* vs *common and vague*. It only sees counts and proportions.

So you add the **global signal** (IDF) to say:

- “train” matches too many docs, so it shouldn’t carry much weight
- “odissa” matches very few docs, so it should carry a lot of weight

Then cosine measures similarity **after** that weighting, which makes the ranking much safer.

One important nuance though:

- Even **TF-IDF + cosine** isn’t perfect. A doc can still win for weird reasons (spammy repetition, synonyms not matching, etc.).
- But compared to TF-only cosine, it’s usually a big improvement because it uses both:
 - **local importance** (TF in that doc)
 - **global distinctiveness** (IDF across the corpus)

So your conclusion is correct: you need that **global comparison** to bake in rarity, otherwise cosine can sometimes prefer a doc that matches common words well but misses the truly identifying term.

make this entire info in word pls from top when i asked u qa question till here pls .

Thought for 1m 53s

Done — I put everything into a clean Word document.